INF1500 – logique des systèmes numériques

Laboratoire 4

Introduction au langage VHDL

Simulation et implémentation d'un circuit en logique combinatoire en VHDL

Révisions :
Alexy Torres Aurora Dugo - V1.0
Jean-Baptiste Lefoul - V1.1
Alexy Torres Aurora Dugo - V1.2
Alexy Torres Aurora Dugo - V1.3
Jean-Baptiste Lefoul - V1.4
Ihsane Majdoubi – V1.5
Yanis Halouani – V1.6

Automne 2026

Département de génie informatique et de génie logiciel
École Polytechnique de Montréal

POLYTECHNIQUE MONTRÉAL

1 Objectifs

L'objectif de ce laboratoire est de concevoir, simuler et implémenter un circuit en logique combinatoire sur la carte FPGA du laboratoire en utilisant le langage de description de matériel VHDL. L'objectif de ce laboratoire est de se familiariser au langage de description matériel VHDL. La validation des différentes composantes sera effectuée à travers la simulation et l'implémentation du circuit complet.

2 Système à réaliser

Dans ce laboratoire, on vous demande de réaliser un circuit qui intègre :

- Cplmt2, un module qui applique l'opération de complément à 2 sur l'entrée A
- BitShift, ce module effectue un décalage logique vers la gauche.
- Mux31_5b, un multiplexeur 3 vers 1 permettant de choisir entre les deux modules et l'entrée du circuit. Le multiplexeur prends en entrée 3 bus de 5 bits, a une entrée de sélection sur 2 bit et a une sortie sur 5 bits.
- Bin2Dec, un module qui effectue un conversion décimale de l'entrée et ressort les unités et les dizaines codées en binaire dans les sorties S0 et S1 respectivement.
- Mux21_4b, un multiplexeur 2 vers 1 permettant de choisir entre les unitées et les dizaines. Le multiplexeur prends en entrée 2 bus de 4 bits, a une entrée de sélection sur 1 bit et a une sortie sur 4 bits.
- Bin7Seg, un module permettant de convertir un nombre binaire en entrée en un signal permettant d'afficher ce nombre sur un afficheur 7 segments.
- GestAffichage, un module qui vous est fourni, permettant d'utiliser plusieurs afficheur 7-segments simultanément, comme pour le laboratoire précédent. Vous n'avez pas à comprendre comment il fonctionne et vous ne devez pas vous en inspirer pour réaliser vos modules.

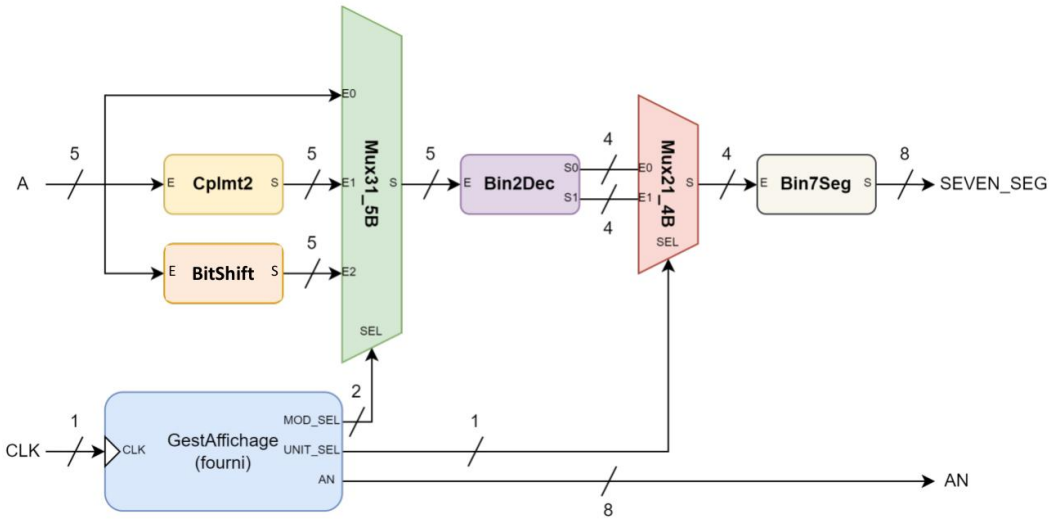


Figure 1 – Schéma du circuit à réaliser

Tout au long de ce laboratoire, vous ne devrez utiliser que les descriptions de flot de donnée et structurel présentes en VHDL. Toute description comportementale sera comptée comme 100% fausse.

3 Cplmt2

Ce module est le même que vous avez réalisé dans le laboratoire précédent, mais avec plus de bits. Exemple : le complément à 2 de 11011 est : 00101.

Pour réaliser ce module, n'utilisez pas de table de vérité, Karnaugh ou équation. Votre tâche est d'utiliser les structures VHDL pour l'implémenter. *Indice : vous pouvez utiliser la bibliothèque* ***IEEE.NUMERIC_STD.ALL***

4 BitShift

Un décalage logique vers la gauche est une opération qui déplace les bits d'un nombre binaire vers la gauche ou vers la droite, en remplissant les nouvelles positions créées par des zéros. Ici on demande un bitShift d'une position vers la gauche. Par exemple : Exemple : si E = 00010110 (22) et qu'on décale de 1 vers la gauche, le résultat est S = 00101100 (44).

Pour réaliser ce module, n'utilisez ni table de vérité, ni carte de Karnaugh, ni équation logique. Les opérations arithmétiques de base (+, -, *) sont pas permises aussi. Votre tâche consiste à implémenter le module en VHDL en utilisant uniquement les structures du langage. Indice : vous pouvez utiliser la bibliothèque IEEE.NUMERIC_STD.ALL.

5 Mux31_5b

Comme d'habitude, le multipleceur va sélectionner l'entrée en fonction des bits de sélections :

SEL	S

00	E0
01	E1
10	E2
11	0

Lors de la validation de ce module, vous aurez **131072 (2^{17})** possibilités. Il va donc falloir trouver un moyen plus simple de le valider. Dans ce cas particulier, vous pouvez appliquer une horloge sur une des entrées seulement, c'est celle que vous testez, et mettre toutes les autre entrées du MUX à la valeur *UNDEFINED* (remove force dans vivado). Il faudra cependant répéter l'opération pour chaque valeurs de *SEL*. Ce test sera donc semi-exhaustif.

Pour réaliser ce module, n'utilisez pas de table de vérité, Karnaugh ou équation. Votre tâche est d'utiliser les structures VHDL pour l'implémenter.

6 Bin2Dec

Dans ce module il faut effectuer la conversion binaire vers décimale. Exemple si E = 11001 ce qui correspond à 25 en décimal alors S0 = 0101 (qui représente 5) et S1 = 0010 (qui correspond à 2).

Pour réaliser ce module, n'utilisez pas de table de vérité, Karnaugh ou équation. Votre tâche est d'utiliser les structures VHDL pour l'implémenter. *Indice : vous pouvez utiliser la bibliothèque* ***IEEE.NUMERIC_STD.ALL***

7 Mux21_4b

Comme d'habitude, le multipleceur va sélectionner l'entrée en fonction des bits de sélections :

SEL	**S**
0	E0
1	E1

Comme pour le module *Mux31_5b*, lors de la validation de ce module, vous aurez beaucoup de possibilités : **512 (2^9)** possibilités. Donc vous pouvez utiliser la même technique de test semi-exhaustif pour faire la validation.

Pour réaliser ce module, n'utilisez pas de table de vérité, Karnaugh ou équation. Votre tâche est d'utiliser les structures VHDL pour l'implémenter.

8 Bin7Seg

Ce module est le même que vous avez réalisé au TP précédent, mais vous allez utiliser une autre méthode pour le faire. Cette fois ci, pour les valeurs au dessus de 9 vous mettrez la sortie à 11111111 (=0xFF).

Pour réaliser ce module, n'utilisez pas de table de vérité, Karnaugh ou équation. Votre tâche est d'utiliser les structures VHDL pour l'implémenter.

9 Afficheur 7 segments

Voici un rappel de comment l'afficheur 7 segments fonctionnent sur la carte. Un afficheur 7 segments contient 7 segments allumables et un point.

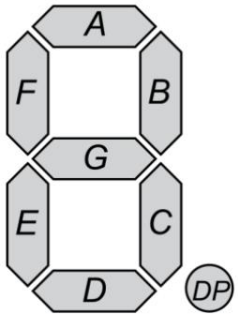


Figure 2 – Afficheur 7 segments

L'afficheur 7 segments comporte 8 entrées, nommées A, B, C,..., G et DP comme montré dans la figure 2 [1].

Chaque bit d'entrée permet d'allumer un segment. Dans ce laboratoire, la sortie de votre circuit doit être un bus de 8 bits. Nous décidons donc de la convention suivante : le bit de poids faible (0) de votre bus correspond au bit A de l'afficheur et le bit de poids fort (7) correspond à DP.

Ainsi si nous voulons afficher "1" nous allons allumer les segments B et C, la valeur de sorties sera donc 11111001. En effet, les segments s'allument quand ils reçoivent un signal bas.

9.1 Validation du circuit complet

Comme au dernier TP, vous devez générer le bitstream et tester votre circuit sur la carte.

10 Fichier de contrainte

Pour le fichier de contrainte voici les ports que vous devez utiliser :

```
set_property -dict { PACKAGE_PIN E3 IOSTANDARD LVCMOS33 } [get_ports {CLK}]
set_property -dict { PACKAGE_PIN T10 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[0] }];
set_property -dict { PACKAGE_PIN R10 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[1] }];
set_property -dict { PACKAGE_PIN K16 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[2] }];
set_property -dict { PACKAGE_PIN K13 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[3] }];
set_property -dict { PACKAGE_PIN P15 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[4] }];
set_property -dict { PACKAGE_PIN T11 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[5] }];
set_property -dict { PACKAGE_PIN L18 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[6] }];
set_property -dict { PACKAGE_PIN H15 IOSTANDARD LVCMOS33 } [get_ports { SEVEN_SEG[7] }];
set_property -dict { PACKAGE_PIN J17 IOSTANDARD LVCMOS33 } [get_ports { AN[0] }];
set_property -dict { PACKAGE_PIN J18 IOSTANDARD LVCMOS33 } [get_ports { AN[1] }];
set_property -dict { PACKAGE_PIN T9 IOSTANDARD LVCMOS33 } [get_ports { AN[2] }];
set_property -dict { PACKAGE_PIN J14 IOSTANDARD LVCMOS33 } [get_ports { AN[3] }];
set_property -dict { PACKAGE_PIN P14 IOSTANDARD LVCMOS33 } [get_ports { AN[4] }];
set_property -dict { PACKAGE_PIN T14 IOSTANDARD LVCMOS33 } [get_ports { AN[5] }];
set_property -dict { PACKAGE_PIN K2 IOSTANDARD LVCMOS33 } [get_ports { AN[6] }];
```

[1] https://en.wikipedia.org/wiki/File:7_Segment_Display_with_Labeled_Segments.svg

```
set_property -dict { PACKAGE_PIN U13 IOSTANDARD LVCMOS33 } [get_ports { AN[7] }];
```

Pour l'entrée A, à vous de placer ses bits sur les commutateurs comme à votre habitude.

11 Questions

1. Donnez deux avantages d'utiliser les structure spéciales VHDL plutôt que d'écrire les équations de circuit.
2. Expliquez la différence entre le type std_logic_vector et le type unsigned ou signed en VHDL.

12 Travail à effectuer

Le travail à réaliser est le suivant :

— Décrire chaque module demandé en VHDL.

— Décrire le circuit au complet en VHDL.

— Écrire le fichier de contrainte pour votre circuit final.

Pour chaque étape, il est ÉVIDENT qu'il faille valider le fonctionnement de son circuit par simulation. Simulez chaque module indépendemment! N'oubliez pas de décrire cette étape dans votre rapport!

Le fichier de contrainte ainsi que l'implémentation de ce circuit est noté.

13 Livrables attendus

Les livrables suivants sont attendus :

— Un rapport écrit format PDF pour le laboratoire (1 point sera retiré si le format n'est pas respecté).

— Une archive ZIP (et non RAR ou autre) de fichiers VHDL au complet.

— Le fichier de contraintes .xdc de votre circuit.

— Le bitstream généré .bit

Veuillez rendre ces fichiers de manière séparée et non dans une archive.

Le rapport doit contenir le titre et numéro du laboratoire, les noms et matricules des coéquipiers ainsi que votre groupe.

Un exemple et un modèle de rapport à respecter sont disponibles sur le site Moodle.

Consultez le site Moodle du cours pour la date et l'heure limites de remise des fichiers.

14 Barème

La note final du laboratoire est sur 7 et est obtenue en faisant la moyenne de deux notes :

— Une note d'évaluation fonctionnel du circuit rendu sur 7 : évalue si le circuit rendu respecte bien les spécifications données

— Une note pour le rapport rendu sur 7 (le barème est détaillé dans le modèle)

10% des points sont retranchés par jour de retard